

Rate Limiter

Сравнение алгоритмов ограничения частоты запросов

Проблематика и задачи

Rate Limiter — механизм контроля интенсивности входящего трафика.

Зачем он нужен в высоконагруженных системах?

- Защита инфраструктуры от перегрузок и DDoS-атак.
- Справедливое распределение ресурсов между клиентами.
- Контроль квот для публичных API.

Цель проекта: Разработать библиотеку на C++23, которая решает эти задачи максимально быстро, потокобезопасно и без утечек памяти (TTL механизмы).

Алгоритм: Token Bucket (Корзина токенов)

Алгоритм накапливает “токены” со скоростью r до максимума b . Каждый запрос стоит 1 токен.

Плюсы:

- $O(1)$ по памяти (храним только время и счетчик).
- Отлично справляется с пиковыми нагрузками (bursts) — накопленные токены позволяют мгновенно пропустить пачку легитимных запросов.

Минусы:

- При неправильно заданном b внезапный всплеск может “пробить” защиту и перегрузить бэкенд.

Алгоритм: Leaking Bucket (Протекающее ведро)

Работает как строгая FIFO-очередь с константной скоростью обработки r . Если очередь переполнена, новые запросы отбрасываются.

Плюсы:

- Идеально сглаживает трафик.
- Выдает стабильный, предсказуемый поток запросов на сервер.

Минусы:

- При резком пике очередь забивается старыми запросами.
- Нет гарантий по времени задержки (latency зависит от позиции в очереди).

Алгоритм: Sliding Window Log (Скользящий журнал)

Хранит временные метки (timestamps) **каждого** запроса в скользящем окне W .

Плюсы:

- Идеальная математическая точность.
- Нет “слепых зон” и скачков на границах временных интервалов.

Минусы:

- $O(N)$ потребление памяти (на каждого клиента нужен свой контейнер/куча).
- Дорогой пересчет при каждом новом запросе — падение пропускной способности при жесткой конкуренции.

Потокобезопасность и Check-then-act

В многопоточной среде параллельные запросы к одному ключу вызывают гонку данных. Обычного `std::atomic` недостаточно из-за составных операций (проверка + списание).

Решение:

- Глобальный `std::shared_mutex` для защиты всей хеш-таблицы ключей.
- Опциональный `std::mutex` на каждый ключ (через `StoredData`), чтобы параллельные запросы к **“user123”** корректно ждали друг друга, не блокируя **“user456”**.

Оптимизация памяти: False Sharing

Глобальный мьютекс становится узким горлышком. Мы применили **горизонтальное шардирование** (ShardedWrapper).

Аппаратная проблема: Если данные разных шардов лежат рядом, ядра процессора инвалидируют кэш друг друга (False Sharing).

Как решили: Использование кастомного аллокатора и выравнивание памяти (LCM от размера кэш-линии и `alignof`), чтобы гарантированно разнести независимые шарды по разным кэш-линиям.